

09 · APIs e integraciones

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **5e91229**

Fecha de generacion: **2026-08-27**

Generado a partir de 09-apis-and-integrations.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Inventario de interfaces
2. La CLI como API
3. Utilidades de macOS consumidas
4. Integración con el proveedor de IA
5. Notificaciones del sistema
6. Integración con GitHub
7. Distribución
8. Lo que el producto deliberadamente no integra

Qué interfaces expone el sistema, qué servicios externos consume y bajo qué condiciones. RootCause **no expone ningún servidor ni endpoint HTTP propio**: sus «APIs» son la línea de comandos, el JSON de salida y el conjunto de utilidades de macOS que consume.

1. Inventario de interfaces

Interfaz	Dirección	Protocolo	Autenticación	Estado
CLI	Entrada	Argumentos de proceso	La del sistema operativo	Activa
JSON de salida	Salida	Archivo / <code>stdout</code>	—	Activa
Reporte Markdown	Salida	Archivo	—	Activa
SQLite propio	Ambas	Archivo local	Permisos del sistema de archivos	Activa
Utilidades de macOS	Salida→Entrada	Proceso hijo	La del usuario que ejecuta	Activa
<code>TCC.db</code>	Entrada	SQLite en solo lectura	Acceso total al disco	Activa, condicionada
Proveedor de IA	Salida	HTTPS vía <code>curl</code>	Bearer token en cabecera	Opcional, apagada
Notificaciones de macOS	Salida	<code>osascript</code>	—	Activa, condicionada
GitHub Actions	—	—	Token del repositorio	Activa
GitHub Pages	Salida	HTTPS	—	Activa
Homebrew cask	Salida	—	—	Plantilla

No existe: servidor HTTP, WebSocket, gRPC, cola de mensajes, webhook entrante, API REST, socket de escucha ni IPC con otros procesos.

2. La CLI como API

Es la interfaz de integración pensada para automatización. Contrato completo en [05 · Referencia técnica](#); aquí van sus propiedades como API.

2.1 Contrato

Propiedad	Valor
Invocación	<code>rootcause <comando> [banderas]</code>
Salida legible	<code>stdout</code> , texto tabulado en español
Salida estructurada	<code>stdout</code> , JSON con formato, con <code>--json</code>
Errores	<code>stderr</code> , texto
Códigos	<code>0</code> correcto · <code>1</code> error de ejecución · <code>2</code> uso incorrecto
Concurrencia	Cada invocación es un proceso independiente

Idempotencia	Los comandos de consulta sí; <code>--accept</code> , <code>clean-caches</code> <code>--yes</code> y <code>kill</code> no
--------------	--

2.2 Comandos con salida JSON

`status`, `snapshot`, `history`, `incidents`, `audit`, `persistence`, `security`, `xprotect`, `tcc`, `connections`, `network`, `events`, `config` `show` y `ai` `explain-latest`.

La forma del JSON es la serialización directa de los modelos de `src/models.rs` con `serde`. Eso significa que **el esquema del JSON es el esquema de los modelos**: añadir un campo a un struct lo añade a la salida.

2.3 Ejemplos de integración

Comprobar en un script si Gatekeeper está activo:

```
rootcause security --json | /usr/bin/python3 -c \
'import json,sys; d=json.load(sys.stdin); print(next(c["status"] for c in d if c["id"]=="gatekeeper"))'
```

Fallar un script si hay algún control de seguridad en crítico:

```
rootcause security --json | grep -q '"severity": "Critical"' && exit 1
```

Guardar una captura completa fechada:

```
rootcause snapshot --output "$HOME/capturas/rootcause-$(date +%Y%m%d-%H%M).json"
```

Revisar los incidentes del último arranque en formato legible:

```
rootcause incidents 5
```

REQUIERE VALIDACIÓN: no existe en el repositorio ningún ejemplo de integración con herramientas externas (SIEM, Jamf, Munki); los de arriba son construcciones de este análisis basadas en el contrato real del CLI.

2.4 Estabilidad

NO DOCUMENTADO EN EL REPOSITORIO: no hay compromiso explícito de estabilidad del formato JSON entre versiones. Los modelos usan `#[serde(default)]` en los campos añadidos, lo que sugiere intención de compatibilidad hacia atrás al **leer**, pero nada garantiza que un campo no se renombre. En la versión 0.1.0 conviene tratar el esquema como inestable.

3. Utilidades de macOS consumidas

Es la «API» real del producto: veinte binarios del sistema. Todos se invocan con **ruta absoluta**, lo que evita que un **PATH** manipulado sustituya una utilidad por otra cosa.

Binario	Argumentos	Módulo	Qué aporta	Si falla
<code>/bin/ps</code>	<code>-axo</code> <code>pid=, user=, command=</code>	<code>macos.rs</code>	Usuario y línea de comandos	Tabla vacía; procesos sin usuario
<code>/usr/sbin/lsof</code>	<code>-i -n -P -FpLftPnT</code>	<code>macos.rs</code>	Conexiones por proceso	Sección Conexiones vacía
<code>/usr/bin/codesign</code>	<code>-dvv <ruta></code>	<code>macos.rs</code>	Firma de código	<code>CodeSignature::Unknown</code>
<code>/usr/sbin/spctl</code>	<code>--status</code>	<code>security.rs</code>	Gatekeeper	Control «Desconocido» en amarillo
<code>/usr/bin/csrutil</code>	<code>status</code>	<code>security.rs</code>	SIP	Idem

/usr/bin/fdesetup	status	security.rs	FileVault	Idem
/usr/libexec/ ApplicationFirewall /socketfilterfw	--getglobalstate, --getstealthmode	security.rs	Firewall y modo encubierto	Respaldo por defaults
/usr/bin/defaults	read /Library/ Preferences/com. apple. alf globalstate	security.rs	Respaldo del firewall	Control «Desconocido»
/bin/launchctl	list	launchd.rs	Servicios cargados y SSH	Sin estado de servicios
/usr/bin/crontab	-l	launchd.rs	Tareas cron del usuario	Sin tareas cron
/usr/bin/osascript	-e <script>	launchd.rs, macos.rs	Login items y notificaciones	Sin login items ni notificaciones
/usr/sbin/arp	-a -n	macos.rs	Vecinos de red	Sección Red vacía
/sbin/route	-n get default	macos.rs	Interfaz y puerta de enlace	Sin contexto de red
/sbin/ifconfig	<interfaz>	macos.rs	IP y MAC locales	Sin IP propia
/sbin/ping	-c 1 -t 1 - q <ip>	macos.rs	Barrido activo	Sin descubrimiento
/usr/bin/ dscacheutil	-q host -a ip_ address <ip>	macos.rs	DNS inverso	Sin nombres de host
/usr/bin/log	show --last Nm -- style compact -- predicate ...	macos.rs	Eventos de seguridad	Sección de eventos vacía
/usr/sbin/sysctl	-n <clave>	macos.rs	Modelo y CPU	Campos vacíos en Acerca
/usr/bin/sw_vers	-productVersion, -productName, -buildVersion	macos.rs	Versión de macOS	Campos vacíos
/usr/bin/id	-un, -u	macos.rs	Usuario y UID	Respaldo 501
/bin/kill	-TERM <pid>	macos.rs	Finalizar proceso	Error explícito
/usr/bin/open	-R <ruta>	macos.rs	Revelar en Finder	Error explícito
/usr/bin/which	<programa>	macos.rs	Comprobar disponibilidad	Se asume ausente
/usr/bin/curl	--silent --show- error --fail --max -time N -X POST ...	ai.rs	Petición al proveedor IA	Error explícito

3.1 Formato de las respuestas que se parsean

Utilidad	Forma esperada	Fragilidad
spctl --status	Contiene assessments enabled o assessments disabled	Media: si Apple cambia el texto, pasa a «Desconocido»

<code>csrutil status</code>	Contiene <code>status: enabled / disabled / custom</code>	Media
<code>fdsetup status</code>	Contiene <code>FileVault is On / Off</code>	Media
<code>socketfilterfw --getglobalstate</code>	Contiene <code>state = 0/1/2</code>	Media, con respaldo
<code>socketfilterfw --getstealthmode</code>	<code>stealth mode enabled</code> o <code>stealth mode is on</code> (ambas aceptadas)	Baja: ya contempla dos redacciones
<code>arp -a -n</code>	<code>? (ip) at mac on iface ...</code>	Baja: parseo posicional tolerante
<code>lsof -F</code>	Líneas <code><letra><valor></code>	Baja: formato estable y diseñado para máquinas
<code>codesign -dvv</code>	Líneas <code>Authority=...</code> , <code>Signature=...</code>	Baja para los casos cubiertos
<code>launchctl list</code>	<code>PID Status Label</code>	Baja
<code>log show --style compact</code>	<code><hora> <tipo> <actividad> <pid></code> <code><proceso>: <mensaje></code>	Media

Los casos de severidad media están cubiertos por la regla «desconocido nunca es verde»: si el formato cambia, el producto lo dice en vez de fingir que todo está bien.

4. Integración con el proveedor de IA

4.1 Condiciones para que exista

Las tres, en orden:

1. `ai.enabled = true` en la configuración.
2. `ai.endpoint` no vacío.
3. La variable de entorno cuyo nombre indica `ai.api_key_env_var` (por defecto `ROOTCAUSE_AI_API_KEY`) existe.

Con los valores de fábrica **ninguna se cumple** y el código no toca la red.

4.2 Petición

Aspecto	Valor
Método	<code>POST</code>
URL	La configurada en <code>ai.endpoint</code>
Cabeceras	<code>Content-Type: application/json, Authorization: Bearer <clave></code>
Cuerpo	Por <code>stdin</code> de <code>curl (--data @-)</code>
Tiempo máximo	<code>ai.timeout_secs</code> (25 s por defecto)
Reintentos	Ninguno . Un fallo se devuelve como error
Transporte	<code>curl --silent --show-error --fail</code>

Cuerpo (estructura real generada por `build_payload`, con valores ficticios):

```
{
  "model": "gpt-4.1-mini",
  "temperature": 0.1,
  "response_format": { "type": "json_object" },
  "messages": [
    {
      "role": "system",
      "content": "Eres un analista forense de macOS. Explicas hallazgos con precisión y sin alarmismo..."
    },
    {
      "role": "user",
      "content": "Incidente detectado en un Mac.\nTítulo: ...\nTipo: ...\nResumen: ..."
    }
  ]
}
```

El **content** del mensaje de usuario es una sola cadena; desplegada en líneas, con un incidente de ejemplo ficticio, dice exactamente esto:

```
Incidente detectado en un Mac.
Título: Persistencia nueva detectada
Tipo: persistence-change
Resumen: LaunchDaemon 'com.ejemplo.helper' apareció respecto a la baseline conocida.
Hipótesis local: algo instaló, modificó o quitó un mecanismo de arranque automático
Evidencia: Ubicación: /Library/LaunchDaemons/com.ejemplo.helper.plist |
           Comando: /usr/local/bin/helper | Cambio: NUEVA

Devuelve SOLO un objeto JSON con las claves: summary (string),
probable_causes (array de strings), suggested_actions (array de strings),
confidence (string: alta|media|baja), warnings (array de strings).
```

4.3 Respuesta esperada

Compatible con la API de chat de OpenAI. El código navega `choices[0].message.content`, que debe contener a su vez un JSON con esta forma:

```
{
  "summary": "Un LaunchDaemon nuevo ejecuta un binario desde una ruta de usuario.",
  "probable_causes": ["Instalación reciente de software", "Persistencia añadida por un tercero"],
  "suggested_actions": ["Comprobar el origen del binario", "Revisar la fecha del plist"],
  "confidence": "media",
  "warnings": ["No se puede confirmar intención maliciosa solo con estos datos."]
}
```

`probable_causes`, `suggested_actions`, `confidence` y `warnings` son opcionales (`#[serde(default)]`): si el modelo devuelve menos campos, no rompe. `summary` es obligatorio.

4.4 Errores

Situación	Mensaje	Código CLI
IA desactivada	La integración IA está desactivada en la configuración	1
Sin endpoint	«Falta <code>ai.endpoint</code> en la configuración»	1
Sin clave	No existe la variable de entorno ... con la API key	1
<code>curl</code> falla o el proveedor responde $\neq$ 2xx	El proveedor IA respondió con error: <code><stderr></code>	1
Respuesta no es JSON	La respuesta del proveedor IA no es JSON válido	1

Falta <code>choices[0].message.content</code>	«La respuesta IA no trae <code>choices[0].message.content</code> »	1
Contenido con forma inesperada	El contenido devuelto por la IA no tiene la forma esperada	1

En todos los casos el CLI añade: *RootCause sigue funcionando con normalidad sin ella*. El incidente local ya persistido **no se toca**.

4.5 Qué se registra de la integración

`update_incident_ai` guarda el consejo dentro del `payload_json` del incidente, y la auditoría registra `ai-explain-latest` con el `incident_id` y el resumen. El campo `provider` se deduce del `host` del endpoint (`provider_from_endpoint`), no de la URL completa: se deja constancia de a dónde se envió sin guardar rutas ni parámetros.

4.6 Límites y dependencias del proveedor

Aspecto	Estado
Control de tarifa / cuotas	No implementado . Cada invocación es una petición
Reintento con espera	No implementado
Caché de respuestas	No implementado ; sí se guarda el consejo en el incidente
Coste	A cargo del usuario, con su propia clave
Modelo por defecto	<code>gpt-4.1-mini</code> (<code>config.ai.model</code>)
Compatibilidad	Cualquier proveedor con API de chat estilo OpenAI: REQUIERE VALIDACIÓN para cada uno

5. Notificaciones del sistema

Aspecto	Detalle
Mecanismo	<code>osascript -e 'display notification "..." with title "..."'</code>
Cuándo	Captura con alerta crítica y <code>alerting.notify_on_critical = true</code>
Contenido	<code>RootCause · <título></code> y el detalle de la alerta
Saneado	Las comillas dobles se sustituyen por simples antes de componer el script
Fallo	Silencioso: si el usuario deshabilitó las notificaciones, no pasa nada
Anti-repetición	<code>notification_cooldown_secs</code> existe en la configuración pero no se usa

Esa última fila es un hallazgo: con una condición crítica persistente y refresco de 5 segundos, la notificación puede repetirse en cada captura. Se registra en [15 · Riesgos](#).

6. Integración con GitHub

6.1 Workflows

Workflow	Disparo	Permisos	Acciones de terceros
<code>ci.yml</code>	push/PR a <code>main</code> , manual	<code>contents: read</code>	<code>actions/checkout@v7</code> , <code>dtolnay/rust-toolchain@stable</code> , <code>Swatinem/rust-cache@v2</code> , <code>actions/upload-artifact@v7</code>
<code>release-macos.yml</code>	tags <code>v*</code> , manual	<code>contents: write</code>	Las anteriores + <code>softprops/action-gh-release@v3</code>
<code>deploy-landing.yml</code>	push a <code>main</code> en <code>landing/**</code> , manual	<code>contents: read</code> , <code>pages: write</code> , <code>id-token: write</code>	<code>actions/configure-pages@v6</code> , <code>actions/upload-pages-artifact@v5</code> , <code>actions/deploy-pages@v5</code>

Los permisos son mínimos por workflow: solo el de release puede escribir en el repositorio. `markdownlint-cli2` se ejecuta con `npm --yes`, es decir, **se descarga en cada ejecución** sin versión fijada: se anota como riesgo de cadena de suministro en [11 · Seguridad](#).

6.2 Publicación de la release

`scripts/release-product.sh --publish` usa la CLI `gh` para etiquetar y publicar. El propio script comprueba seis condiciones antes de tocar el remoto (**REQUIERE VALIDACIÓN**: no se ejecutó en este análisis).

7. Distribución

Canal	Estado	Detalle
Compilación desde el código	Recomendado	<code>cargo build --release</code>
<code>.dmg</code> de la release de GitHub	Activo	Sin firmar ni notarizar
<code>RootCause-app.zip</code>	Activo	Bundle comprimido
<code>SHA256SUMS.txt</code>	Activo	Integridad de los artefactos
Homebrew cask	Plantilla	<code>packaging/homebrew/rootcause.rb</code> , sin tap publicado
Mac App Store	No	Incompatible con lo que hace el producto: INFERENCIA
crates.io	No	<code>publish = false</code> en <code>Cargo.toml</code>

8. Lo que el producto deliberadamente no integra

No hace	Por qué (según el propio repositorio)
Telemetría o analítica	«Todo el análisis es local»
Comprobación de actualizaciones	Implicaría una salida a red constante
Envío de informes de fallo	Idem

Servidor local de administración	Fuera del alcance de la edición de escritorio
Aplicar reglas de firewall	<code>block-ip</code> entrega el comando; ejecutarlo es decisión del usuario
Eliminar archivos sospechosos	«Diagnóstico primero. Intervención después»
Instalarse como LaunchAgent	Una herramienta que vigila la persistencia ajena no añade la suya en silencio

Siguiente lectura recomendada: [10 · Configuración](#).